Bau eines RISC-V-Prozessors mit einem FPGA

Eine Jahresarbeit

Verfasser: Tim Heilmann

Betreuerin/Betreuer: Fr. Michaelis

Institution: Rudolf-Steiner-Schule Schloß Hamborn

Abgabedatum: 24.02.2025

Inhaltsverzeichnis

1. Ideenfindung . . . 3
2. Einleitung . . . 4
   2.1. Theoretischer und praktischer Kontext . . . 4
   2.2. Projektziel und Abgrenzung . . . 4
   2.3. Gliederung der Arbeit . . . 5
3. Grundlagen der Instruktionsverarbeitung . . . 6
   3.1. Instruktionen . . . 6
   3.2. Programme . . . 7
   3.3. Zusammenfassung . . . 8
4. Emulator . . . 9
   4.1. Funktion und Nutzen des Emulators . . . 9
   4.2. Funktionsprinzip . . . 9
   4.3. Implementierung . . . 10
   4.4. Test und Validierung . . . 11
   4.5. Zusammenfassung des Kapitels . . . 11
5. Hardware-Entwurf und Implementierung . . . 12
   5.1. Prozessorarchitektur . . . 12
   5.2. Finite State Machine . . . 12
   5.3. Datenpfad . . . 14
   5.4. FPGA . . . 15
      5.4.1. Funktionsweise . . . 15
      5.4.2. Prozessorimplementierung . . . 16
      5.4.3. Test und Validierung . . . 18
6. Peripheriegeräte . . . 20
   6.1. Memory-Mapped I/O . . . 20
   6.2. LED-Matrix und Framebuffer . . . 20
   6.3. Game-Controller . . . 21
7. Anwendungen . . . 22
   7.1. Snake-Spiel . . . 24
8. Fazit und Ausblick . . . 26
   8.1. Zusammenfassung . . . 26
   8.2. Reflexion . . . 26
   8.3. Ausblick und Erweiterungsmöglichkeiten . . . 27
   8.4. Danksagung . . . 27
9. Glossar . . . 28
Quellen . . . 30

1. Ideenfindung

Ursprünglich wollten ein Freund und ich ein Videospiel mit dem Kernthema „Rudolf Steiner“ programmieren. Wir hatten uns bereits informiert und eine Grafikdesignerin gefunden. Außerdem hatten wir schon genaue Vorstellungen davon, wie das fertige Produkt aussehen sollte. Über mehrere Wochen planten wir und hatten mithilfe von Tutorials bereits erste Testspiele erstellt. Nach etwa anderthalb Monaten stellten wir jedoch fest, dass wir beide kein ausreichendes Interesse mehr an dem Projekt hatten, da der Arbeitsaufwand erheblich gewesen wäre und im Verhältnis zu dem, was in etwa sechs Monaten realistisch durchführbar gewesen wäre, zu groß war. Wir entschieden uns daher, zwei separate Projekte anzugehen. Mehrfach wurde uns geraten, dass es ein waghalsiges Unterfangen sei, ein Spiel zu programmieren. Wir waren anfangs überzeugt, das Projekt auf einen präsentablen Stand bringen zu können. Diese Annahme erwies sich als falsch.

Hier zeigte sich bereits eine Tendenz zur Selbstüberschätzung, die sich in gewissem Maße auch in der zweiten Projektwahl widerspiegelt. Ich programmiere gerne, mag logisches Denken und bin auch einigermaßen talentiert sowie definitiv interessiert. Dennoch gab es stets eine Wissenslücke, die mich störte: Ich wusste, wie man in Python programmiert, und hatte Grundkenntnisse in C, einer sehr hardwarenahen Programmiersprache. Hardwarenah bedeutet, dass die Sprache wenige Abstraktionen nutzt. Dennoch war mir nicht klar, wie ein Computer auf elementarer Ebene funktioniert. Mir war unbegreiflich, wie ein Computer ein Display, eine Tastatur oder den Speicher ansteuert. Ich hatte keine Ahnung von Instruktionen; jedoch war mir bekannt, dass es Assembler gibt und wie dessen grobe Struktur aussieht. Von Instruktionssatzarchitektur und Rechnerarchitektur im Allgemeinen hatte ich jedoch keine Kenntnisse.

Während der Planung des Spiels hatte ich immer noch eine alternative Projektidee: den Bau einer einfachen *Central Processing Unit (CPU)*, welche auf einem programmierbaren Elektronikbaustein implementiert wird. Diese Idee stammte von meinem Vater. Nach der Entscheidung, die Spielidee nicht weiterzuverfolgen, griff ich sie auf. Das neue Projekt versprach, Wissenslücken zu schließen, die in den letzten Jahren entstanden waren. Im Laufe der Projektdurchführung kamen noch einige Aspekte hinzu, zum Beispiel ein Display und ein Controller. Auch die Idee für den sozialen Teil, ein *Snake*-Turnier, entstand im Verlauf des Projekts durch Beratung mit meiner Mentorin und meinem Vater.

2. Einleitung

In diesem Kapitel werden das Ziel dieser Arbeit definiert und Hintergrundinformationen zum Thema aufgezeigt. Zudem wird die Struktur der Arbeit vorgestellt.

2.1. Theoretischer und praktischer Kontext

Der Prozessor ist der zentrale Bestandteil eines Computers und interagiert mit verschiedenen Komponenten, wie Speicher sowie Ein- und Ausgabegeräten. Er führt elementare Befehle aus, die die Ausführung von Programmen ermöglichen.

Computer und somit Prozessoren sind heutzutage fest im Alltag etabliert und ein essenzieller Teil des Arbeitslebens vieler Menschen. Sie bilden das technologische Fundament zahlreicher Industrien und werden von nahezu allen auch im privaten Bereich genutzt. Trotz ihrer beinahe konstanten Präsenz verstehen nur wenige Menschen die Funktionsweise von Computern ausreichend. Die Grundlage für dieses Verständnis bildet ein spezielles Teilgebiet der Informatik: die *Rechnerarchitektur*, die sich mit dem Aufbau und der Funktionsweise von Computersystemen befasst. Zu den prägendsten Persönlichkeiten dieses Fachgebiets gehören John L. Hennessy und David A. Patterson, deren Werke Teil des Kernlehrplans an vielen Universitäten sind, insbesondere „Computer Architecture: A Quantitative Approach“ [1] und „Computer Organization and Design: RISC-V Edition“ [2], die für diese Arbeit die wesentliche Grundlage bilden. Ein zentraler Aspekt der Rechnerarchitektur ist der sogenannte Instruktionssatz. Dieser stellt die definierte Schnittstelle zwischen dem Programmierer und der Hardware dar und definiert die verfügbaren Befehle des Prozessors. Er ist das Unterscheidungsmerkmal zwischen den verschiedenen Prozessoren, zum Beispiel Intel, ARM oder die M-Serie von Apple. Viele dieser Instruktionssatzarchitekturen sind proprietär und sehr komplex.

Eine öffentlich zugängliche, frei verfügbare Alternative ist die RISC-V *Instruction Set Architecture (ISA)*. RISC steht für Reduced Instruction Set Computer; das bedeutet, dass die ISA so klein wie möglich gehalten ist. Das „V“ steht für die Generation, in diesem Fall fünf. RISC-V ist modular aufgebaut: Ein minimaler Kern, der in dieser Arbeit umgesetzt werden soll, kann durch zahlreiche optionale Erweiterungen ergänzt werden [3]. Johnson beschreibt praktische Implementierungen und Performance-Aspekte [4]. Für die praktische Umsetzung und das Testen von Programmen auf dem Emulator und später auf dem FPGA sind weitere Werke wie Patterson & Waterman [5] sowie Gay [6] und Smith [7] hilfreich.

2.2. Projektziel und Abgrenzung

Das Ziel meines Projekts ist der Entwurf und die Implementierung eines RISC-V-Prozessors mit einem FPGA. RISC-V, oder auch RV, ist im Kontext dieses Projekts eine geeignete ISA, da sie reduziert und zudem modular ist. Es gibt eine Basisversion, die jeder RISC-V-Prozessor implementieren muss. Darüber hinaus gibt es Erweiterungen, welche die ISA beispielsweise um dedizierte Multiplikationsbefehle ergänzen. Zudem ist RV ein realer Standard und keine für didaktische Zwecke vereinfachte Befehlssatzarchitektur. Außerdem ist RV unter der *Creative-Commons-Lizenzierung* lizenziert und ist *Open Source Software (Open Source)*. Die Lizenz besagt, dass man den Code abändern, wiederveröffentlichen und kommerziell nutzen darf, ohne Lizenzgebühren zu zahlen. Dass der Standard Open Source ist, bedeutet, dass alles, wie die Spezifikationsdokumente, öffentlich zugänglich ist. Weiterhin beinhaltet das Ziel des Projekts die Ansteuerung und Nutzung von Peripheriegeräten, in diesem Fall eines Displays und eines Game-Controllers, sowie die Durchführung eines Snake-Turniers.

Es ist nicht Ziel dieser Arbeit, einen hochoptimierten Prozessor zu bauen. Auch das Spiel wurde bewusst einfach gehalten, um die Funktion des Displays und des Controllers zu veranschaulichen und zu zeigen, dass reale C-Programme auf dem Prozessor ausführbar sind.

2.3. Gliederung der Arbeit

Die Arbeit ist in mehrere Kapitel gegliedert. Als Erstes wird eine kurze Einführung in die allgemeine Funktionsweise von Prozessoren und die Umsetzung in eine Prozessorarchitektur präsentiert, danach folgen in weiteren Kapiteln der Entwurf und die Umsetzung des RISC-V-Prozessors. Dabei wird der *Emulator* vorgestellt, welcher das Verhalten der CPU simuliert. Darauf folgt die Beschreibung des Hardware-Entwurfs des Prozessors, einschließlich Prozessorarchitektur, Datenpfad und Finite State Machine. Anschließend werden die Integration und Nutzung von *Peripheriegeräten* – hier spezifisch ein Display und ein DualShock-Game-Controller – zusammen mit dem Konzept *Memory Mapped I/O (MMIO)* erläutert. Im letzten Kapitel werden das C-Programm Snake sowie der soziale Teil, das Snake-Turnier, beschrieben, gefolgt von einer abschließenden Reflexion und Bewertung des Projekts inklusive der Danksagung.

3. Grundlagen der Instruktionsverarbeitung

In diesem Kapitel wird das grundlegende Funktionsprinzip des Prozessors erläutert. Da Prozessoren heutzutage sehr weit entwickelt sind und deshalb ohne Vorkenntnisse nicht tiefgehend verstanden werden können, beschränkt sich diese Darstellung auf die grundlegenden Konzepte. Als Erstes wird die Funktion und Ausführung von einzelnen Instruktionen vorgestellt und erläutert, wie sich diese unter Zuhilfenahme von Kontrollflussinstruktionen zu einem Programm kombinieren lassen. Diese Grundlagen sind für das Verständnis der folgenden Kapitel *Emulator* und *Hardware* notwendig.

3.1. Instruktionen

Prozessoren führen Instruktionen (Befehle) aus, welche die Ausführung von Programmen ermöglichen. Die Bedeutung und Ergebnisse dieser Instruktionen werden über die *ISA* festgelegt. Beispiele dafür sind das Addieren von zwei Zahlen oder logische Operationen, wie das Vergleichen von zwei Zahlen. Außerdem gibt es noch Lade- und Speicherbefehle, um Daten zwischen Hauptspeicher und Prozessor hin- und her zu kopieren. Zudem kann mit Kontrollflussinstruktionen die Reihenfolge der Ausführung der Instruktionen gesteuert werden, beispielsweise um Wiederholungen oder Verzweigungen im Programmfluss zu realisieren.

Jedoch kann ein Prozessor mit der textuellen Darstellung von Programmcode und Instruktionen nichts anfangen. Deshalb gibt es Programme, welche den für Menschen lesbaren Code in Instruktionen übersetzen (sog. Compiler), die anschließend von weiteren Programmen (Assemblern) in für den Prozessor nutzbaren Maschinencode umgewandelt werden.

In Abbildung 1 wird anhand eines Additionbefehls dargestellt, wie Instruktionen in einem Prozessor abgearbeitet werden. Der Prozessor verfügt über:

- Instruktionsspeicher - zum Speichern des Programms
- Hauptspeicher - zum Speichern der Daten
- Register - eine kleine Menge von internem Speicher, aus der die Operanden für Befehle gelesen und Resultate von Operationen gespeichert werden
- Rechenwerk - führt Operationen auf den Daten in den Registern durch
- Steuerwerk - kontrolliert den Ablauf

Das Beispiel zeigt, wie die Anweisung `a = b + c` in der Programmiersprache C für den Prozessor in kleinere Operationen zerlegt und ausgeführt wird. Die Werte für die Variablen *b* und *c* sind am Anfang im Hauptspeicher abgelegt. Um mit diesen Daten rechnen zu können, müssen sie zuerst in Register geladen werden, was mit den *lw*-Befehlen (load word to register) erfolgt. Der Inhalt der Variablen *b* und *c* wird in die Register *x1* bzw. *x2* kopiert. Anschließend wird die Addition mit der Instruktion *add x3, x1, x2* durchgeführt. Dabei werden die in Register *x1* und *x2* kopierten Variablen addiert und das Ergebnis in Register *x3* gespeichert. Das Resultat wird dann mit dem *sw*-Befehl wieder zurück in den Hauptspeicher kopiert.

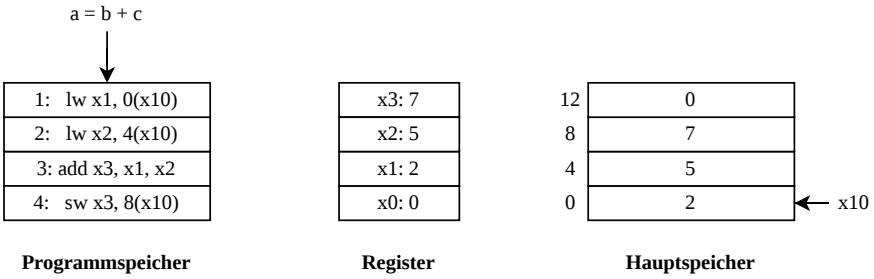

Abbildung 1: Veranschaulichung einer Addition mit Assembler

3.2. Programme

Mit arithmetischen Operationen alleine lassen sich jedoch nur einfache Programme umsetzen. Deshalb gibt es neben den arithmetischen auch noch Kontrollflussinstruktionen, welche datenabhängige Sprünge und Verzweigungen ermöglichen. Die Notwendigkeit und Verwendung dieser Instruktionen wird anhand eines Beispiels illustriert, bei dem die Summe aller natürlichen Zahlen von 1 bis 10 addiert wird (siehe Listing 1). Dazu wird eine Schleife (*while*) mit einer Zählvariable (*i*) benötigt, deren Zustand überprüft werden muss. Dies erfolgt über die Kontrollflussinstruktionen. Bei jedem Durchlauf der Schleife wird der bisherige Wert der Zählvariable zum Wert der Summe addiert und die Zählvariable auf die nächste natürliche Zahl gesetzt. Sobald die obere Grenze von 10 überschritten wird, wird die Schleife beendet. Dieses datenabhängige Verhalten wird mittels verschiedener Kontrollflussinstruktionen umgesetzt.

In Abbildung 2 wird der Kontrollfluss grafisch dargestellt; Listing 2 zeigt die Umsetzung in RISC-V-Assembler. Im Assembler-Programm sind noch weitere Typen von Instruktionen zu sehen:

- Die `li`-Instruktion (load immediate) lädt einen numerischen Wert, der in der Instruktion selbst angegeben wird, in ein Register. Da der numerische Operand unmittelbar in der Instruktion kodiert ist, wird er *Immediate*-Wert genannt.
- Die `addi`-Instruktion (add immediate) ist ebenfalls eine Immediate-Instruktion, sie verhält sich wie eine `add`-Instruktion, wobei ein Operand ein *Immediate*-Wert ist.
- Die `bgt`-Instruktion (branch if greater than) vergleicht zwei Register und springt zum angegebenen Sprungziel, wenn der erste Operand größer ist als der zweite.
- Bei LABEL: handelt es sich nicht um eigentliche Prozessorinstruktionen, sondern um Sprungmarken, welche im Assemblercode verwendet werden können, um Sprungziele zu markieren.

```c
int sum = 0;
int i = 1;

while (i <= 10)
{
  sum = sum + i;
  i = i + 1;
}
```

Listing 1: Berechnung der Summe aller natürlichen Zahlen von 1 bis 10 in C

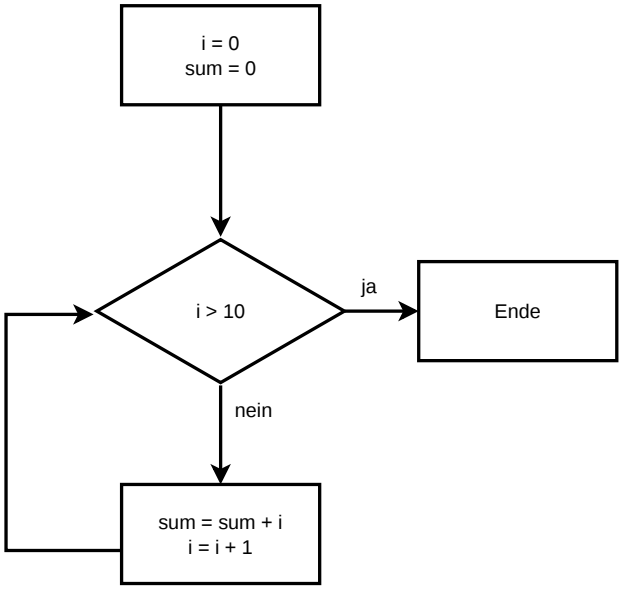

Abbildung 2: Kontrollflussdiagramm eines Programms, welches alle Zahlen von 1 bis 10 zusammenzählt

```assembler
START:
  li x1, 0  # sum
  li x2, 1  # i
  li x3, 10 # maximaler Wert von i
LOOP:
  bgt x2, x3, END # wenn i > 10 springe zu END
  add x1, x1, x2  # sum = sum + i
  addi x2, x2, 1  # i = i + 1
  j LOOP          # springe zu LOOP
END:
  ...
```

Listing 2: Berechnung der Summe aller natürlichen Zahlen von 1 bis 10 in RISC-V-Assembler

3.3. Zusammenfassung

In diesem Kapitel wurden die Grundlagen der Instruktionsverarbeitung anhand verschiedener Beispiele erläutert. Hier ist jedoch anzumerken, dass moderne Prozessoren über diese Instruktionen hinaus noch über komplexere Befehle verfügen, zum Beispiel gibt es Instruktionen für Division, Multiplikation, Vektoren und Statusregister. Außerdem gibt es noch Instruktionen für die Implementierung eines Betriebssystems. Diese dienen dazu, zwischen einem unprivilegierten Modus mit eingeschränkten Rechten für Nutzerprogramme und einem privilegierten Modus mit vollen Rechten für das Betriebssystem hin und her zu wechseln.

4. Emulator

In diesem Kapitel wird der Emulator vorgestellt, welcher die Ausführung von Programmen im RISC-V-Instruktionssatz in Software simulieren kann. Zudem werden noch zwei Test-Frameworks vorgestellt, die die korrekte Funktionsweise des Emulators überprüfen können.

4.1. Funktion und Nutzen des Emulators

Der Emulator ist eine Software, die das Verhalten von Hardware imitiert *(lat. aemulārī, „nachahmen“)*. In diesem Fall imitiert der Emulator das Verhalten des RISC-V-Prozessors, der die RISC-V-Basis-ISA RV32I implementiert. RISC-V ist eine Load–Store-Architektur, bei der ausschließlich spezielle Lade- und Speicherinstruktionen auf den Hauptspeicher zugreifen. Arithmetische und logische Operationen arbeiten ausschließlich auf Registern. Register sind ein interner Speicher, auf den der Prozessor sofort zugreifen kann. Dieses Prinzip vereinfacht den Datenpfad und erleichtert sowohl die Emulation als auch den späteren Hardware-Entwurf. Der Emulator ist in C geschrieben, was diverse Vor- und Nachteile hat. C ist hardwarenah, das bedeutet, dass die Sprache wenige Abstraktionen besitzt. Dies ist ein Vorteil, da es die Übertragung vom Emulator zum Hardware-Entwurf vereinfacht. Andererseits ist die Hardwarenähe auch ein Nachteil, da die fehlenden Abstraktionen den Aufwand und die Codegröße erhöhen. Trotzdem bietet C fast volle Kontrolle über Speicher, Optimierung und die Ausführung des gesamten Programms. Für den Emulator wird eine Abstraktionsschicht geschaffen, welche die RISC-V-Instruktionen in die des *Host* umwandelt. Die RV32I-Implementierung des Emulators entspricht der Implementierung auf dem FPGA in architektonischer Hinsicht, berücksichtigt allerdings Aspekte wie *Pipelining*, *Taktzyklen* oder parallele Ausführung nicht.

Die Entwicklung des Emulators ist ein wichtiger Schritt in Richtung des Hardware-Entwurfs, da sie es zulässt, sich mit der Architektur auseinanderzusetzen und diese zu implementieren, ohne mit FPGA-spezifischen Problemen konfrontiert zu werden. Zudem kann der Emulator die Sequenz der Instruktionsausführung und die Register- und Speicherinhalte protokollieren. Dieser sogenannte Ausführungs-Trace kann zur Validierung des Hardware-Entwurfs genutzt werden, siehe Abschnitt 4.4.

4.2. Funktionsprinzip

Instruktionen werden sequenziell ausgeführt. Sie durchlaufen fünf Phasen, welche zyklisch wiederholt werden (vgl. Abbildung 3):

- **Fetch (IF)** In der Fetch-Phase wird die Instruktion aus dem Speicher geholt.
- **Decode (ID)** In der Decode-Phase wird sie dekodiert. Das heißt, es wird der Typ der Instruktion ermittelt (arithmetisch-logische Instruktionen, Lese-/Speicheroperation, Kontrollflussinstruktion). Zudem werden Quell- und Zielregister aus der Instruktion gelesen und eventuell Immediatewerte extrahiert. Register sind ein eigener kleiner Speicher, der direkt von der CPU genutzt werden kann; es gibt 32 dieser Register.
- **Execute (EX)** In der Execute-Phase werden die arithmetischen und logischen Operationen ausgeführt und Speicheradressen berechnet.
- **Memory (MEM)** In der Memory-Phase werden Daten in den Speicher geschrieben oder aus diesem gelesen.
- **Writeback (WB)** In der Writeback-Phase wird das in der EX-Phase berechnete Ergebnis bzw. die aus dem Speicher gelesenen Daten ins Zielregister geschrieben. Weiter wird die in der nächsten IF-Phase gelesene Instruktion ausgewählt. Dazu wird der *Program Counter (PC)* um 4 inkrementiert, außer es handelt sich um eine *Kontrollflussinstruktion*, welche den PC auf das Sprungziel setzen kann.

Als Informationsgrundlage für die Implementierung dient die „RISC-V Reference Card“ [8], welche einen Überblick über alle Instruktionen und deren Format bietet.

Der Emulator und der spätere Hardware-Entwurf folgen dem Harvard-Speichermodell, welches besagt, dass der Instruktions- und Datenspeicher strikt getrennt sind. Dies ermöglicht es, Instruktionen und Daten gleichzeitig zu laden und zu speichern, was die Geschwindigkeit des Systems stark erhöht.

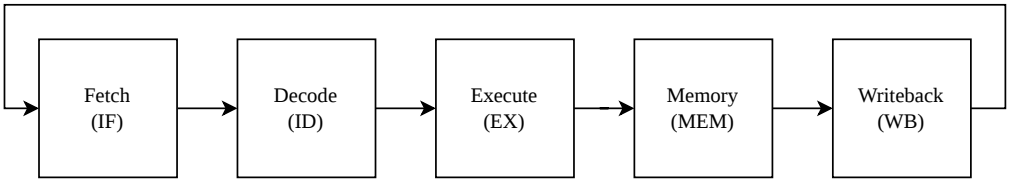

Abbildung 3: Die fünf Phasen der Instruktionsverarbeitung

4.3. Implementierung

Der Emulator besteht aus zwei *Arrays*. Ein Array ist eine Datenstruktur, die mehrere Werte gleichen Typs in nummerierten Speicherplätzen speichert. Das eine Array repräsentiert den Instruktionsspeicher mit 4096 *Wörtern*, das andere den Datenspeicher, ebenfalls mit der gleichen Anzahl an Wörtern. Ein Wort ist im Kontext dieses Projekts eine Zahl oder Speicherstelle, die als 32 Bit bzw. 4 Bytes dargestellt wird. Im Falle der RV32I-Architektur sind die Instruktionen, der *PC* und die Register ein Wort breit, wie die Zahl 32 in der Abkürzung bereits zeigt.

Zu Beginn wird der PC auf 0 gesetzt, um die erste Instruktion aus dem Instruktionsspeicher zu holen (Fetch). Dies erledigt eine dedizierte Funktion, welche als Parameter die Adresse (PC) verwendet und dann die jeweiligen Bytes der Instruktion an diesem Index des Arrays zurückgibt. Die Instruktionen werden, obwohl sie 32 Bit breit sind, byteweise gespeichert, um die Kompatibilität mit anderen Erweiterungen sicherzustellen. Dies ist auch der Grund, warum der Program Counter für den Zugriff auf die direkt folgende Instruktion um 4 erhöht wird ($\mathrm{PC} + 4$). Anschließend wird die Instruktion in die einzelnen Felder dekodiert (Decode), indem zuerst der Typ festgestellt wird und dann die Felder, welche die restlichen Informationen bereitstellen, extrahiert werden. Auf die spezifische Instruktionsdekodierung und Immediate-Generierung wird in Abschnitt 5.2 nochmals detailliert eingegangen. Für das Verständnis dieses Abschnitts ist nur wichtig, dass es einen Opcode gibt, der die Art der Instruktion bestimmt, und dass die restlichen Felder weitere Informationen wie die Operanden (die Register, auf denen die Operation ausgeführt wird) und spezifische Anweisungen, z. B. *add* (addiert zwei Zahlen und speichert sie in einem gegebenen Register), beinhalten. Darauf folgt die Ausführung (Execute) der Instruktion; wenn zum Beispiel die Instruktion *add* ausgeführt wird, werden hier die zwei Zahlen addiert. Im Falle einer Instruktion, die den Speicher nutzt, wird hier die Lade- respektive Speicheradresse berechnet. Kontrollflussinstruktionen berechnen hier auch die nächste Sprungadresse; sie vergleichen zum Beispiel die Werte zweier Register, wie 1 und 200. Aufgrund des Resultats $1 < 200$ kann nun ein Sprung zur Zieladresse erfolgen, indem der PC für die nächste Instruktion entsprechend gesetzt wird. Falls die Instruktion auf den Speicher zugreift, werden in der Memory-Phase Daten aus dem Datenspeicher gelesen oder in diesen geschrieben. In der abschließenden Writeback-Phase werden berechnete Ergebnisse, sofern vorhanden, in die Zielregister zurückgeschrieben und der Program Counter entsprechend aktualisiert. Hier gilt, dass x0, das erste Register, immer den Wert 0 hat.

4.4. Test und Validierung

Während des Programmierens des Emulators gab es eine Vielzahl von *Bugs*. Die beiden gravierendsten waren jedoch eine fehlerhafte Immediatedekodierung und eine inkorrekte Bytereihenfolge bei *Load*- und *Store*-Befehlen. Um diese Fehler zu finden und anschließend zu korrigieren, gibt es verschiedene Ansätze, unter anderem *Unit-Tests* und *Execution Tracing*. Bei einem Unit-Test wird ein kleiner Teil eines Systems isoliert getestet, deshalb ist es auch wichtig, Code modular zu halten. Für jeden Test wird ein Input und ein erwarteter Output definiert. Der Code wird mit dem Test-Input getestet und die Ausgabe wird mit dem erwarteten Output verglichen. *Execution Tracing* beschreibt einen Prozess, bei dem während der Programmausführung Informationen über dessen internen Zustand gesammelt werden, um Fehler zu finden.

Die Immediatedekodierung ist essenziell für die korrekte Ausführung von Instruktionen, da sie die unmittelbaren Werte extrahiert, die in Instruktionen eingebettet sind. Fehler bei diesem Prozess können zu unvorhersehbarem Verhalten führen, da unter anderem Sprungadressen mit diesen Werten berechnet werden. Die fehlerhafte Bytereihenfolge bei *Load*- und *Store*-Befehlen führt dazu, dass Daten nicht korrekt im Speicher gespeichert oder aus diesem gelesen werden. Um diese Fehler zu finden und zu korrigieren, habe ich Unit-Tests für die Immediatedekodierung und die Speicherbefehle geschrieben. Zusätzlich habe ich die Werte, die in den Registern gespeichert wurden, während der Ausführung von Programmen inspiziert, um sicherzustellen, dass sie den erwarteten Werten entsprechen. Nach der Korrektur dieser Fehler verhielt sich der Emulator wie erwartet und konnte die Testprogramme korrekt ausführen.

4.5. Zusammenfassung des Kapitels

In diesem Kapitel wurde der Emulator vorgestellt, der das Verhalten eines RISC-V-Prozessors simuliert. Es wurden die grundlegenden Funktionen und die Architektur des Emulators erläutert, inklusive der fünf Phasen der Instruktionsverarbeitung: Fetch, Decode, Execute, Memory und Writeback. Es wurde erläutert, wie der Program Counter funktioniert und warum die Immediatedekodierung essenziell ist. Zudem wurden zwei Debugging-Methoden vorgestellt: *Execution Tracing* und *Unit-Tests*. Es wurde erläutert, wie diese Methoden genutzt wurden, um das Verhalten des Prozessors zu korrigieren und zu validieren.

5. Hardware-Entwurf und Implementierung

In diesem Kapitel werden die Konventionen von RISC-V zusammen mit dem Datenpfad erläutert. Darüber hinaus wird das Konzept der *Finite State Machine (FSM)* vorgestellt und das Funktionsprinzip des FPGAs inklusive der Prozessorimplementierung und des Entwurfs präsentiert. Abschließend werden die Test- und Validierungsmethoden beschrieben, welche zur Sicherstellung der korrekten Funktion des Hardware-Entwurfs genutzt werden.

5.1. Prozessorarchitektur

Der Hardware-Entwurf des Prozessors implementiert die RISC-V-Basis-ISA RV32I. Diese hat einige Konventionen; die wichtigsten sind die Load–Store-Architektur, 32 Register mit einer Breite von 32 Bit und der Standardwert des Registers x0, welches immer den Wert 0 hat. Das Prinzip der Load–Store-Architektur ist, dass nur dedizierte Speicherbefehle auf den Datenspeicher zugreifen dürfen und arithmetische und logische Operationen nur mit Registern durchgeführt werden. Der Speicher folgt dem Harvard-Speichermodell, welches Instruktionsspeicher und Datenspeicher trennt. Der Speicher ist byte-adressiert, was bedeutet, dass jedes Byte einzeln geladen und gespeichert werden kann. Dies verbessert die Effizienz, da manche Werte nicht 32 Bit breit sind und somit Speicherplatz gespart werden kann. Außerdem ermöglicht es die Kompatibilität zwischen der RV32C- und der RV32I-ISA. RV32C ist eine komprimierte ISA, Befehle sind nur 16 Bit, also 2 Bytes, breit. Dies erhöht die Effizienz und reduziert die Codegröße um 40 bis 60 %. Da 32 Bit 4 Byte entsprechen, wird der PC im Fall von RV32I immer auf durch 4 teilbare Werte erhöht. RISC-V nutzt standardmäßig *Little Endian*. Die *Endianness* gibt vor, wie Daten gespeichert werden; im Binärsystem gibt es ein *Least Significant Byte (LSB)* und ein *Most Significant Byte (MSB)*. Das LSB ist das Byte, welches die kleinste Veränderung der Zahl hervorruft, wenn es geändert wird; das MSB verursacht die größte Veränderung. Bei Little-Endian wird das LSB an der niederwertigsten Adresse gespeichert und bei Big-Endian wird das MSB an der niederwertigsten Adresse gespeichert, siehe Abbildung 4. Wie bereits im vorherigen Kapitel erwähnt, folgt der Hardware-Entwurf dem 5-stufigen Ausführungsmodell aus dem Buch von Hennessy & Patterson [1], welches die Instruktion in fünf Phasen verarbeitet: Fetch, Decode, Execute, Memory und Writeback. Dies sorgt für eine klare Trennung der Funktionen und verbessert die Übersichtlichkeit des Designs. Es gibt auch die Möglichkeit, die Instruktion in mehr oder weniger Phasen zu verarbeiten. Jedoch hat sich dieses Modell als Standard etabliert, da es eine gute Balance zwischen Komplexität und Effizienz bietet. Es ermöglicht auch die Implementierung von Pipelining, bei dem mehrere Instruktionen gleichzeitig in verschiedenen Phasen verarbeitet werden, was die Effizienz des Prozessors erhöht. Dies ist jedoch nicht Teil dieses Projekts, da es die Komplexität des Prozessors stark erhöht hätte.

5.2. Finite State Machine

Die im vorherigen Kapitel beschriebene Ausführung der Instruktionen kann als Zustandsautomat *FSM* umgesetzt werden. Die FSM ist ein Konzept aus der theoretischen Infor-

Adresse	Hex	Binärdarstellung	Signifikanz
0x00	0x78	01111000	LSB ← niederwertig
0x01	0x56	01010110	
0x02	0x34	00110100	
0x03	0x12	00010010	höchstwertig → MSB

Abbildung 4: Speicherlayout eines 32-Bit-Wertes (0x12345678) im Little-Endian-Format

matik, findet jedoch auch beim Hardware-Entwurf Anwendung. Eine FSM hat Zustände und Übergänge und ist mit einem Flussdiagramm zu vergleichen. Es gibt einen Anfangszustand, von dem aus die FSM startet, und es gibt Übergänge zwischen den Zuständen, welche abhängig von den Daten sind.

In diesem Fall wird die FSM genutzt, um den Prozessor vereinfacht darzustellen (vgl. Abbildung 6). Jede Verzweigung bei der Ausführung einer Instruktion wird hier als eigener Zustand dargestellt. Wie gewohnt beginnt der Zyklus in der IF-Phase, in welcher die auszuführende Instruktion aus dem Programmspeicher in das Instruktionsregister geladen wird, zudem wird noch der nächste PC um 4 erhöht. Anschließend folgt die ID-Phase, in welcher die Quell- und Zielregister sowie unmittelbare Werte extrahiert werden. Die Kodierung der unterschiedlichen Instruktionstypen ist in Abbildung 5 zu sehen. Nun erfolgt eine Verzweigung abhängig vom Typ der Instruktion, hier werden vereinfacht vier Instruktionstypen unterschieden: LW/SW, Reg/Reg, Reg/Imm und Branch. Im Falle einer LW/SW-Instruktion wird die Lade-/Speicheradresse aus einem Register und dem Immediatewert berechnet und in der folgenden Phase MEM an den Speicher geleitet. Bei der SW-Instruktion werden auch die Daten weitergeleitet. Falls eine `load`-Instruktion ausgeführt wird, kommen in dieser Phase die Daten aus dem Speicher zurück, welche in der Writeback-Phase ins Zielregister geschrieben werden. Bei einer `store`-Instruktion werden keine Daten zurückgegeben. Anschließend wird der PC um 4 erhöht, da kein Sprung erfolgt. Im Falle einer Reg/Reg-Instruktion werden die Registerwerte an das Rechenwerk geleitet, wo die Operation ausgeführt wird. Das Ergebnis wird in der Writeback-Phase ins Zielregister geschrieben. Bei einer Reg/Imm-Instruktion ist der Ablauf ähnlich, jedoch wird hier ein Immediatewert anstatt eines Registerwerts an das Rechenwerk geleitet. Bei einer Branch-Instruktion wird die nächste PC-Adresse abhängig von einem Vergleich zweier Registerwerte berechnet. In der MEM-Phase wird dann entschieden, ob der PC auf die berechnete Sprungadresse oder auf die nächste Instruktion gesetzt wird.

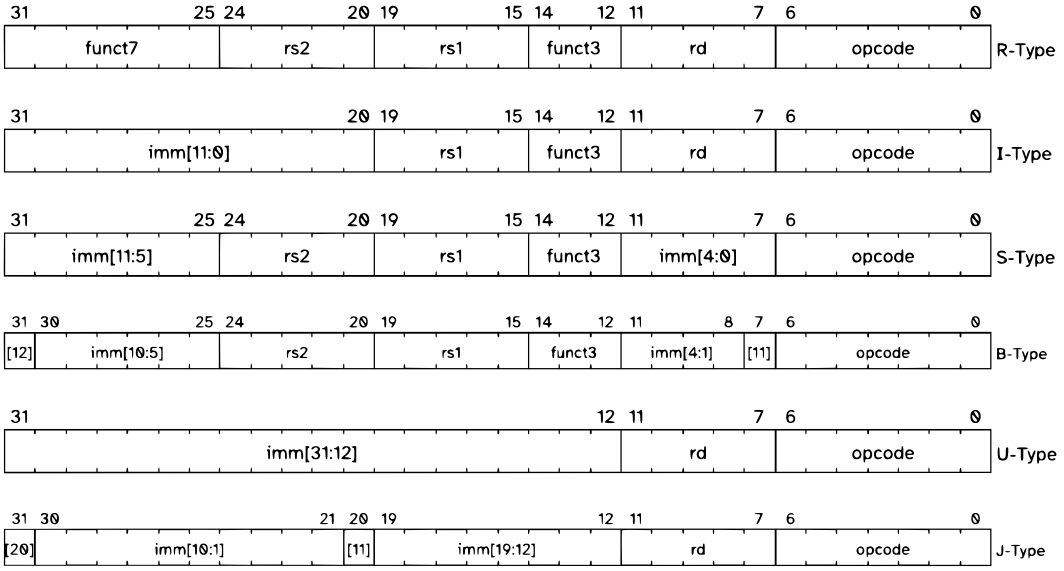

Abbildung 5: Die sechs grundlegenden Instruktionsformate der RISC-V-Befehlssatzarchitektur. Übernommen aus [9], S. 26.

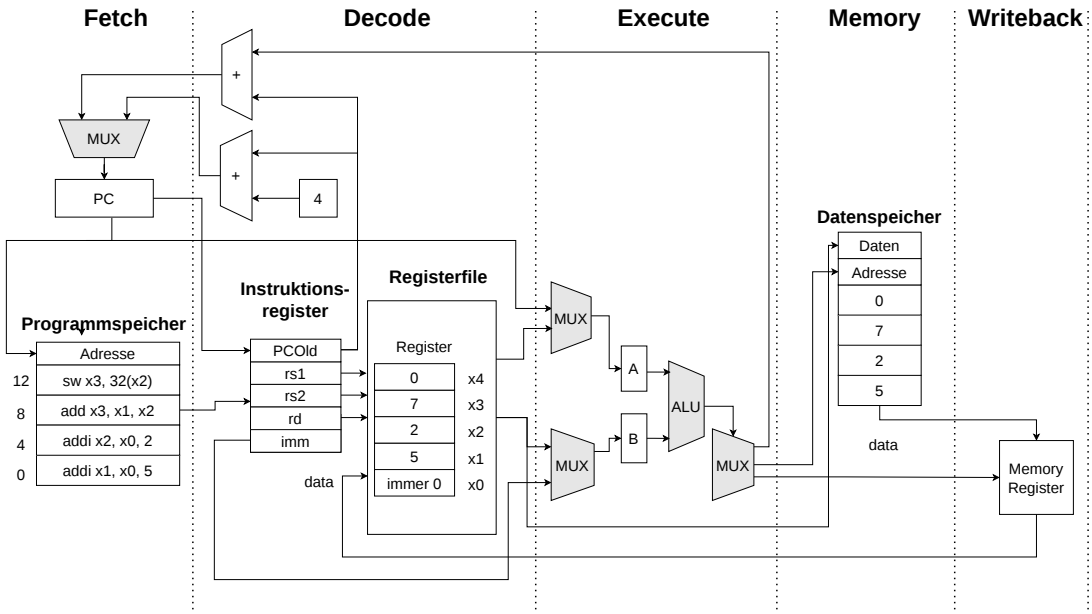

Abbildung 7: Vereinfachte schematische Darstellung des implementierten Datenpfads. Vom Kontrollpfad gesteuerte Elemente werden grau dargestellt

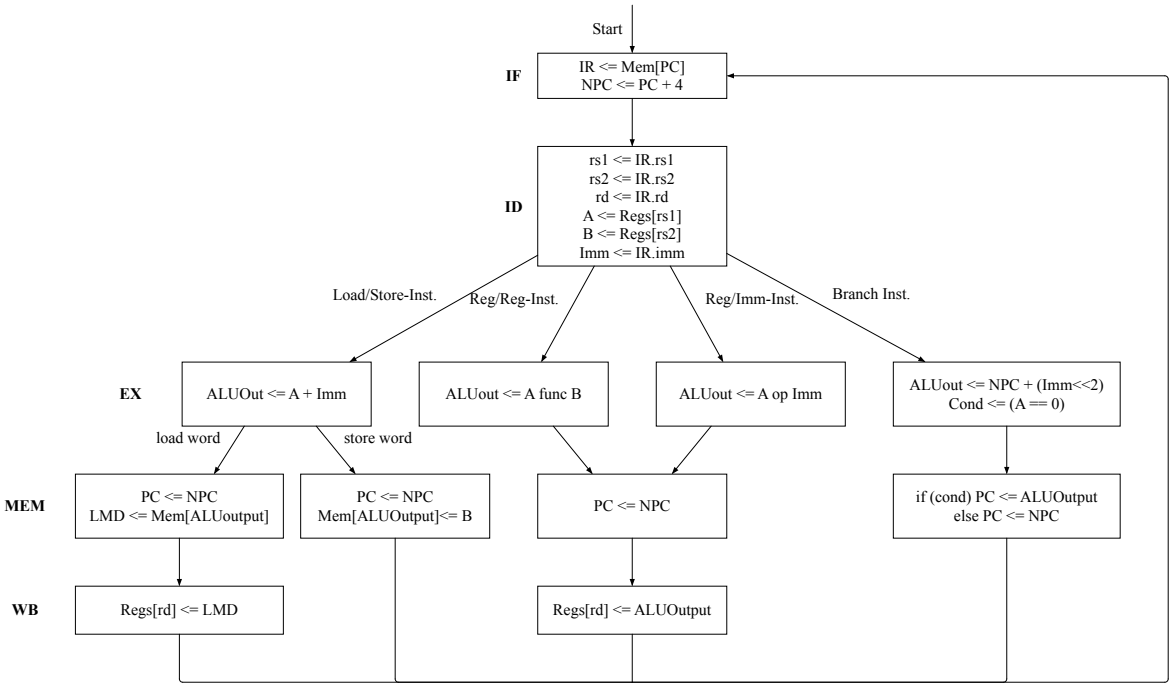

Abbildung 6: Finite State Machine für die Kontrolle der Ausführung

5.3. Datenpfad

Der Datenpfad des Prozessors beschreibt den Fluss von Instruktionen, Operanden und Ergebnissen innerhalb der Hardware über die fünf Ausführungsphasen. Er definiert, wie Signale miteinander verbunden sind und welche Möglichkeiten der Verarbeitung existieren. Um nun eine spezifische Operation durchzuführen, gibt es die Steuereinheit. Die Steuereinheit generiert abhängig vom *opcode*, welcher den Typ der Instruktion angibt, Kontrollsignale. Diese Signale werden an alle in Abbildung 7 grau eingefärbten Einheiten weitergegeben. Die erwähnten Einheiten MUX und ALU haben verschiedene Funktionen. MUX steht für Multiplexer und ist ein Element, welches mehrere Eingänge und Ausgänge haben kann, die über ein Kontrollsignal gewählt werden. Die ALU, oder auch arithme-

tisch-logische Einheit, führt je nach Kontrollsignal eine andere Operation aus; zum Beispiel addiert oder vergleicht sie numerische Werte. Das Gegenstück zum Datenpfad nennt sich Kontrollpfad und generiert Steuersignale, welche die Funktion der Einheiten im Datenpfad steuern, um die gewünschte Instruktion auszuführen. Er entspricht konzeptuell den Abläufen in der Finite State Machine, lässt sich grafisch aber nicht hilfreich darstellen.

5.4. FPGA

5.4.1. Funktionsweise

Ein *Field Programmable Gate Array (FPGA)* ist ein integrierter Schaltkreis, welcher aus vielen rekonfigurierbaren Logikelementen besteht. Mit ihm lässt sich jede elektronische Schaltung nachbauen; diese Vielseitigkeit macht ihn ideal für dieses Projekt. Zusätzlich zu den Logikelementen besitzt er auch integrierte Speicherblöcke, welche für die spätere Implementierung des Prozessors als Programm-, Haupt- und Grafikspeicher genutzt werden. Zusätzlich gibt es Ein- und Ausgabeblöcke, über die die Schaltung auf dem FPGA mit externen Komponenten, wie Displays, verbunden werden kann. Da ein Prozessor nichts weiter als eine digitale Schaltung ist, kann man diesen auch mit dem FPGA umsetzen. Neben dem Kernprozessor können somit auch weitere Module für die Ansteuerung von Peripheriegeräten, in diesem Fall ein Display und ein Game-Controller, direkt integriert werden. Das im Rahmen dieses Projekts benutzte FPGA stammt von der Marke Gowin und heißt: GW5A-LV25MG121NC1/I0.

Um Schaltungen mit dem FPGA zu realisieren, gibt es unterschiedliche Wege. Zum Beispiel ist es möglich, Schaltpläne übersetzen zu lassen oder die Schaltung aus einer abstrakten Programmiersprache (Hardware-Beschreibungssprache) zu generieren. Dieser Ansatz wird in dieser Arbeit verfolgt, da ein Simulator genutzt werden kann, um das Verhalten zu simulieren und zu prüfen, bevor der Entwurf auch auf das FPGA geladen werden kann. Im Rahmen dieser Arbeit wird die Hardware-Beschreibungssprache SystemVerilog genutzt, um den Prozessor zu implementieren. SystemVerilog ist eine der am weitesten verbreiteten Sprachen für diesen Zweck. Um den Code in Logikbausteine zu übersetzen, gibt es Syntheseprogramme. Hier werden die vom Hersteller bereitgestellte Gowin-EDA-Software und die zugehörige Entwicklungsumgebung genutzt. Nach der Synthese wird das Design mit einem Programmierkabel auf das FPGA übertragen. Das FPGA behält seine Konfiguration nur bei, solange es mit Strom versorgt wird. Jedoch kann das Design auch auf einen Speicherchip (FLASH) geladen werden, welcher seinen Inhalt auch ohne Strom beibehält. Sobald das FPGA mit Strom versorgt wird, liest es die Konfiguration, falls vorhanden, aus dem FLASH oder wartet, bis es programmiert wird.

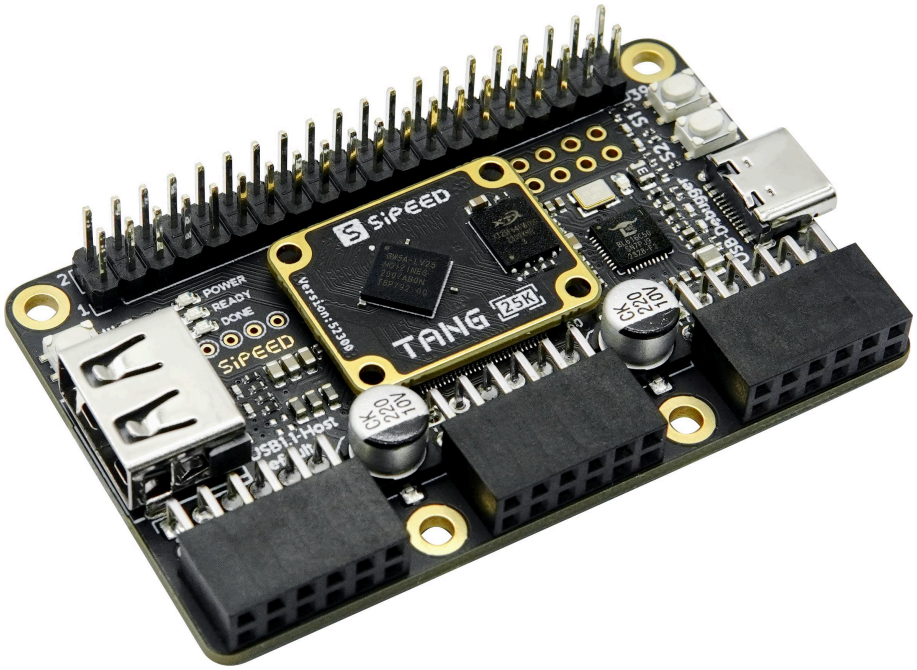

Abbildung 8: Tang Primer 25K Board mit GW5A-LV25MG121NC1/I0 FPGA

In Abbildung 8 ist das in der Arbeit verwendete Entwicklungsboard zu sehen; mittig befindet sich das FPGA. Auf der vom Leser abgewandten Seite befinden sich Pins, an die diverse Komponenten angeschlossen werden können. An der gegenüberliegenden Seite sind plastikummantelte Steckplätze zu sehen, welche für Erweiterungen wie ein Display oder einen Controller gedacht sind. Links und rechts sind USB-Anschlüsse zu sehen. Der rechte ist für die Programmierung zuständig; der linke Anschluss dient ebenfalls als Erweiterungsanschluss.

5.4.2. Prozessorimplementierung

Wie in Abbildung 9 zu sehen ist, ist die Implementierung des Prozessors modular. Dies entspricht auch der SystemVerilog-Code-Struktur. Die Module auf der linken Seite sind Speicherkomponenten, welche den Instruktions-, Grafik- und Datenspeicher für den Prozessor bereitstellen. In der Mitte ist das Herzstück des Prozessors: das Kontroll- und Datenpfadmodul. Dieses implementiert die in den vorherigen Kapiteln beschriebene FSM und bindet alle Komponenten an. Listing 3 zeigt exemplarisch einen Ausschnitt aus der Instruktionsdekodierungslogik als SystemVerilog-Implementierung. Zusätzlich sind noch der LED-Matrix-Controller und der SPI-Controller sichtbar, welche die Schnittstelle zwischen Peripherie und Prozessor bilden und mit den Steckverbindern auf dem Entwicklungsboard verbunden sind.

Insgesamt umfasst die Implementierung der in Abbildung 9 dargestellten Module ca. 1100 Zeilen Code und die Synthese dauert ungefähr 15 Sekunden. Neben der Implementierung sind auch noch umfangreiche Tests vorhanden.

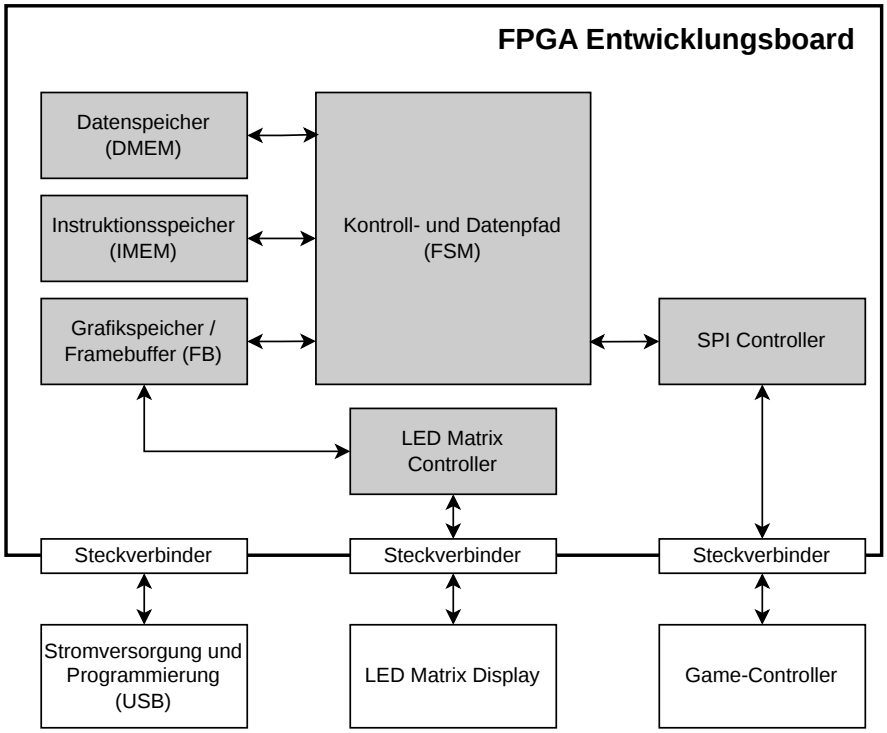

Abbildung 9: Blockdiagramm des entwickelten Prozessors inklusive der Module für die LED-Matrix und den Game-Controller

```System Verilog
DECODE: begin
    opcode <= instr[6:0];
    rd <= instr[11:7];
    funct3 <= instr[14:12];
    rs1 <= instr[19:15];
    rs2 <= instr[24:20];
    funct7 <= instr[31:25];

    case (instr[6:0])
      7'b0010011:     //I type arithmetic
          imm <= {{20{instr[31]}}, instr[31:20]};
      7'b0000011:     //I type load
          imm <= {{20{instr[31]}}, instr[31:20]};
      7'b0100011:     //S type
          imm <= {{20{instr[31]}}, instr[31:25], instr[11:7]};
      7'b1100011:     //B type
          imm <= {{19{instr[31]}}, instr[31], instr[7], instr[30:25],
          instr[11:8], 1'b0};
      7'b1101111:     //J type jal
          imm <= {{11{instr[31]}}, instr[31], instr[19:12], instr[20],
          instr[30:21], 1'b0};
      7'b1100111:     // I type jalr
          imm <= {{20{instr[31]}}, instr[31:20]};
      7'b0110111:     //U type lui
          imm <= {instr[31:12], 12'b0};
      7'b0010111:     //U type auipc
          imm <= {instr[31:12], 12'b0};
      default:
          imm <= 32'b0;
    endcase
    state <= EXECUTE;
  end
```

Listing 3: Immediate-Dekodierungslogik aus dem FSM-Modul, implementiert in System-Verilog

5.4.3. Test und Validierung

Um Fehler im Hardware-Entwurf zu finden, wurde der Verilog-Simulator Icarus-Verilog zusammen mit Testbenches genutzt und die Signale mittels einer Erweiterung namens Surfer inspiziert. Testbenches sind vergleichbar mit Unit-Tests. Sie laden ein Modul, definieren den Test-Input und den erwarteten Output, simulieren das Modul und vergleichen den Output mit dem erwarteten Output. Die Daten, welche während der Simulation generiert werden, können mit dem Waveform-Viewer Surfer visualisiert werden. Dies ermöglicht die Darstellung von Signalen über einen Zeitraum, um so die internen Zustände der Schaltung zu inspizieren.

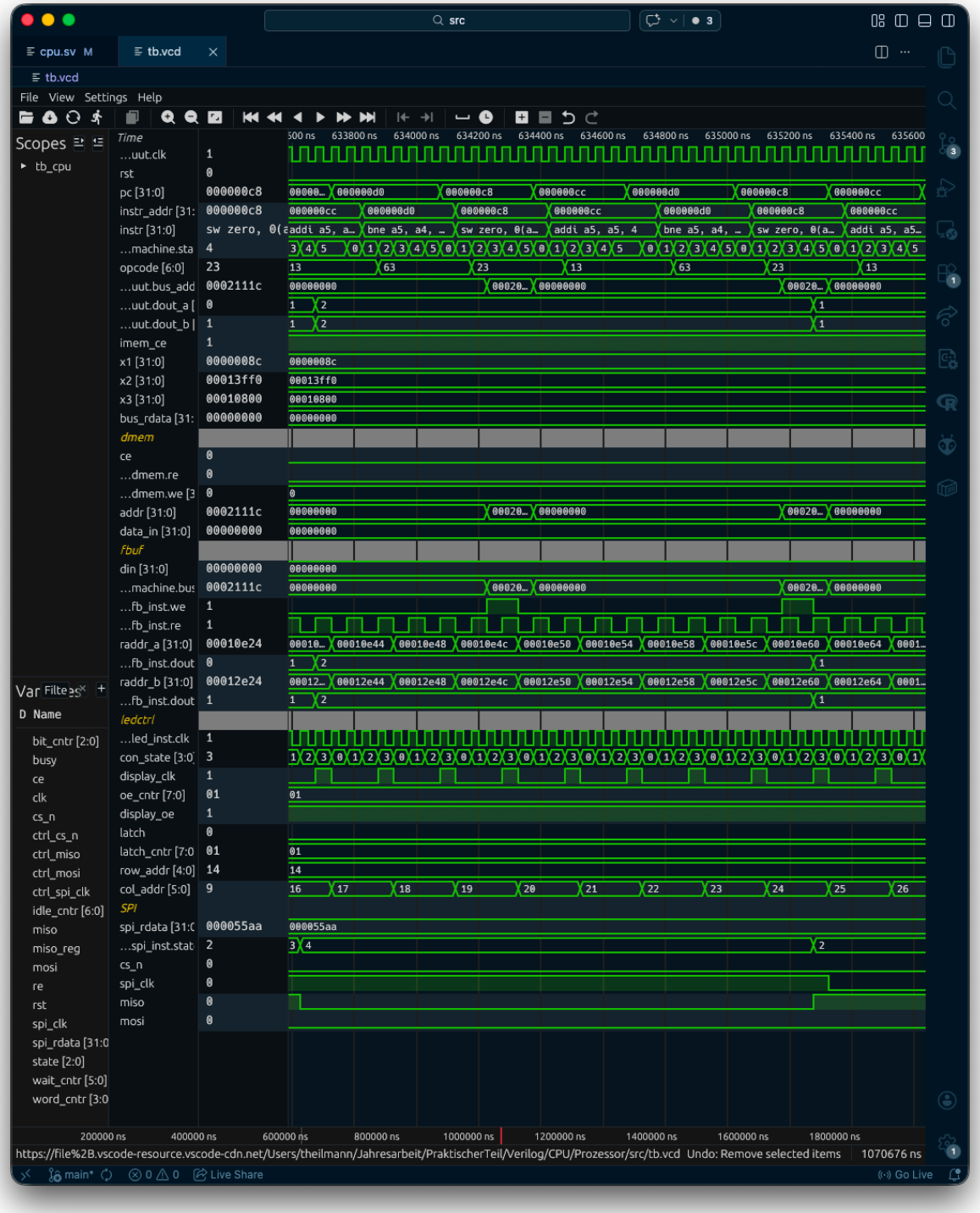

Abbildung 10: Visualisierung von Signalen mit Surfer, hier zu sehen die internen Signale während der Ausführung eines Testprogramms

6. Peripheriegeräte

In diesem Kapitel werden die Grundlagen von Memory-Mapped I/O erläutert und die Funktionsweise der entwickelten Peripheriecontroller kurz beschrieben.

6.1. Memory-Mapped I/O

Memory-Mapped I/O ist eine Methode, um Peripheriegeräte, wie Tastaturen, Game-Controller und Displays, anzusteuern. Dabei werden die Geräte wie ein dedizierter Ort im Speicher behandelt. Wenn zum Beispiel an eine bestimmte Adresse geschrieben wird, leuchtet ein Pixel auf dem Display. Dazu müssen die Module zur Ansteuerung der Peripheriegeräte auf Lese- und Schreibzugriffe reagieren. Durch Schreibzugriffe lässt sich der Zustand der Peripheriegeräte ändern, mit Lesezugriffen kann der Zustand ausgelesen werden. Durch dieses Konzept werden keine speziellen I/O-Befehlserweiterungen benötigt, sondern der Prozessor kann sich auf die existierenden `load`- und `store`-Befehle verlassen. Beispielsweise könnte beim Lesen der Adresse einer Tastatur der ASCII-Code der aktuell gedrückten Taste zurückgeliefert werden.

6.2. LED-Matrix und Framebuffer

Der LED-Matrix-Controller dient dazu, einen Bildschirm mit 64 x 64 LEDs anzusteuern. Jeder Pixel hat drei LEDs in den Farben Rot, Grün und Blau, welche einzeln angesteuert werden können. Somit können neben den Hauptfarben durch gleichzeitiges Aktivieren die Mischfarben Gelb, Weiß, Magenta, Cyan und Schwarz dargestellt werden. Um die Darstellung auf der LED-Matrix zu ändern, muss der LED-Controller die neuen Farbinformationen Pixel für Pixel für eine ganze Zeile zum Display schicken und dieses dann aktualisieren (siehe Abbildung 9).

Dieser Prozess benötigt mehrere Signale, die zwischen Controller und Display nach einem genau definierten zeitlichen Ablauf ausgetauscht werden müssen. Um die Nutzung für Programmierer zu vereinfachen, wurde ein komplett autonomer LED-Display-Controller entworfen. Das darzustellende Bild wird dazu in einem Grafikspeicher (Framebuffer) auf dem FPGA abgelegt, der dann vom Display-Controller ständig ausgelesen und zum Display geschickt wird. Jedem Pixel auf dem Display ist dabei eine Adresse im Framebuffer zugeordnet. Um die Farbe eines Pixels zu ändern, wird der korrespondierende Farbwert als 3-Bit-Sequenz (RGB) in den Framebuffer geschrieben. Das Beispiel in Abbildung 11 zeigt diesen Ablauf.

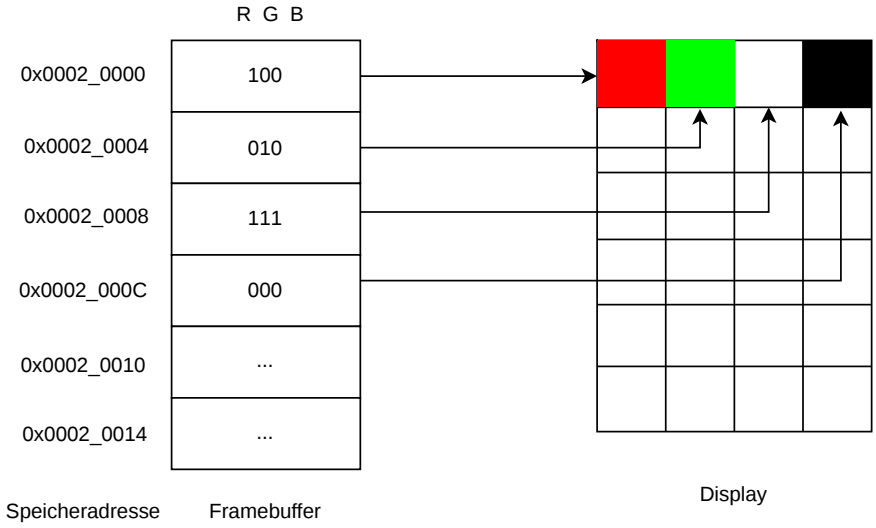

Abbildung 11: Ansteuerung der LED-Matrix über Memory-Mapped I/O, hier zu sehen die Adressierung der einzelnen Pixel über den Framebuffer

6.3. Game-Controller

Da das Ziel der Arbeit die Demonstration eines Computerspiels auf dem Prozessor ist, wird ein Eingabegerät benötigt. In diesem Fall wurde ein Game-Controller gewählt. Dieser Controller nutzt ein standardisiertes Protokoll zur Ansteuerung von Peripheriegeräten, das Serial Peripheral Interface (SPI). Das Protokoll bestimmt, dass ein Befehl an den Controller gesendet wird und dieser dann antwortet. Der Controller hat mehrere Tasten, welche entweder gedrückt oder nicht gedrückt sein können. Der Controller sendet den Status der Tasten als Bit-Sequenz (vgl. Abbildung 12) zurück, welche dann vom SPI-Modul gespeichert wird. Der Prozessor kann diese Sequenz mit `load`-Instruktionen lesen. Wie der Display-Controller ist auch der SPI-Controller autonom. Der Zustand des Controllers kann als 32-Bit-Wort gelesen werden.

Controller Status Register

0x0003_0000

31	Bit 31-16 nicht verwendet	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Bit 15: LINKS – D-Pad-Links-Taste
Bit 14: UNTEN – D-Pad-Unten-Taste
Bit 13: RECHTS – D-Pad-Rechts-Taste
Bit 12: OBEN – D-Pad-Oben-Taste
Bit 11: START – START-Taste
Bit 10: R3 – Taste des rechten Analogsticks
Bit 9: L3 – Taste des linken Analogsticks
Bit 8: SELECT – SELECT-Taste

Bit 7: □ – Quadrat-Taste
Bit 6: ✕ – Kreuz-Taste
Bit 5: ○ – Kreis-Taste
Bit 4: △ – Dreieck-Taste
Bit 3: R1 – Rechte Schultertaste 1
Bit 2: L1 – Linke Schultertaste 1
Bit 1: R2 – Rechte Schultertaste 2
Bit 0: L2 – Linke Schultertaste 2

Abbildung 12: Abfrage des Zustands der Tasten des Game-Controllers

7. Anwendungen

Die Anwendungen dienen dazu, dass der Prozessor einen richtigen Verwendungszweck hat. Außerdem sind sie auch eine Möglichkeit, die Funktion des Prozessors zu validieren.

Um ein Programm auf dem Prozessor ausführen zu können, müssen der Instruktions- und der Datenspeicher mit den richtigen Werten initialisiert werden, d. h., im Instruktionsspeicher muss der Maschinencode stehen und im Datenspeicher die Initialwerte von Datenstrukturen. Die entsprechenden Speicherinhalte werden bei der Synthese vom FPGA-Designtool eingebettet und bei der Konfiguration gleichzeitig mit der Schaltung auf das FPGA übertragen. Da es sich bei dem entwickelten Prozessor um einen RISC-V-Prozessor handelt, können die öffentlich verfügbaren Programmierwerkzeuge für die Übersetzung von Assembler- oder C-Programmen in Maschinencode verwendet werden. Hierzu wurden passende Konfigurationsdateien und Hilfsprogramme entwickelt, mit denen eine automatische Konversion in die vom FPGA akzeptierten Formate erfolgen kann.

Listing 4 zeigt ein Programm, das alle Komponenten miteinander verwendet: den Prozessor, den LED-Controller und den SPI-Controller. Anfangs wird in der Startfunktion (main) der Bildschirm komplett schwarz initialisiert (init_display) und im Zentrum ein einzelner weißer Pixel gesetzt (fb_write). Das Programm liest nun fortlaufend den Status der Tasten des Game-Controllers aus. Wird eine Richtungstaste gedrückt, wird der Pixel um eine Position in der entsprechenden Richtung versetzt. Beim Erreichen des Bildschirmrands taucht er auf der gegenüberliegenden Seite wieder auf.

```c
#define SCREEN_W 64
#define SCREEN_H 64
#define BTN_LEFT  0x8000
#define BTN_DOWN  0x4000
#define BTN_RIGHT 0x2000
#define BTN_UP    0x1000
#define COLOR_BLACK 0u
#define COLOR_GREEN 2u
#define COLOR_RED 4u
#define COLOR_WHITE 7u

volatile unsigned int * const spi = (volatile unsigned int * ) 0x00030000;
volatile unsigned int * const fb  = (volatile unsigned int * ) 0x00020000;
// setze Pixel (row, col) auf color
void fb_write(unsigned row, unsigned col, unsigned color) {
    fb[row * 64 + col] = color;
}
// setze alle Pixel auf Farbe color
void init_display(unsigned color){
    for(int i = 0; i < SCREEN_H; i++){
        for(int j = 0; j < SCREEN_W; j++){
            fb_write(i, j, color);
        }
```

```
    }
}
// Warteschleife
__attribute__((noinline)) void param_delay(unsigned int count) {
    for(unsigned int i = 0; i < count; i++) {
        __asm__ volatile("nop");
    }
}

int main(){
    unsigned int px = 32;
    unsigned int py = 32;
    int dx = 0;
    int dy = 0;
    unsigned int buttons;

    init_display(COLOR_BLACK);

    while(1) {
        // Pixel an
        fb_write(py, px, 7);
        param_delay(30000);
        // Pixel aus
        fb_write(py, px, 0);
        param_delay(30000);
        // Buttons lesen
        buttons = spi[0];

        if(buttons & BTN_LEFT){
            dx = -1; dy = 0;
        } else if(buttons & BTN_RIGHT){
            dx = 1; dy = 0;
        } else if(buttons & BTN_UP){
            dx = 0; dy = -1;
        } else if(buttons & BTN_DOWN){
            dx = 0; dy = 1;
        } else {
            dx = 0; dy = 0;
        }
        px = (px + dx + 64) & 63;
        py = (py + dy + 64) & 63;
    }
    return 0;
}
```

Listing 4: Testprogramm, bei dem LED-Controller, SPI-Controller und Prozessor miteinander integriert sind. Ein einzelner Pixel lässt sich steuern.

7.1. Snake-Spiel

Das ursprüngliche Projektziel war, das klassische Snake-Spiel nachzubauen, um dann ein Turnier zu veranstalten. Snake ist ein einfaches Computerspiel mit dem Ziel, die als Linie von Pixeln dargestellte Schlange mit zufällig platzierten Äpfeln zu füttern. Wenn die Schlange ihren Kopf bewegt, folgt der Rest des Körpers dem Pfad des Kopfes. Beim Verzehr des Apfels nimmt die Schlange an Länge und Geschwindigkeit zu. Gewonnen hat der Spieler, wenn die Schlange das ganze Spielfeld ausfüllt. Verloren wird, wenn der Kopf der Schlange mit ihrem Körper kollidiert oder wenn die Schlange sich außerhalb des Displays begibt. Das FPGA mit angeschlossem Display und angeschlossem Controller ist in Abbildung 13 zu sehen.

Bei der Entwicklung des Prototypen traten bei der Verwendung komplexerer Programm- und Datenstrukturen unerwartete Fehler auf. Es zeigte sich, dass die Ausführung von Funktionen sowie das Laden und Speichern von Daten in bestimmten Grenzfällen fehlerhaft implementiert war. Ein gravierender Fehler bestand darin, dass der Prozessor aus dem Instruktionsspeicher lesen sollte, jedoch stattdessen aus dem Datenspeicher las. Dies führte zum Laden beliebiger Werte. Die Ursache lag im Linker-Skript.

Das Linker-Skript ist eine Konfigurationsdatei, die dem Compiler vorgibt, wie die verschiedenen Sektionen eines Programms im Speicher abgelegt werden. Für diese Erklärung sind folgende Sektionen relevant:

- `.text` - Hier werden die Instruktionen gespeichert und in den Instruktionsspeicher geladen.
- `.rodata` - Hier werden konstante Daten abgelegt, die ebenfalls in den Instruktionsspeicher geladen werden.
- `.data` - Hier werden initialisierte Variablen abgelegt und in den Datenspeicher geladen.

Beim Kompilieren des Snake-Spiels werden die Instruktionen in `.text`, initialisierte Variablen in `.data` und Konstanten in `.rodata` geladen. Da Konstanten unveränderlich sind, ist es sinnvoll, sie im Instruktionsspeicher abzulegen. In der programmierten Implementierung können `load`-Befehle jedoch nur aus dem Datenspeicher lesen. Das Linker-Skript definiert eine Adresse für die Konstanten, die in dieser Implementierung jedoch Teil des Datenspeichers ist, weshalb arbiträre Daten geladen werden.

Da diese Fehler nur in Grenzfällen auftraten, war das Finden und Beheben der Fehler zeitlich äußerst aufwendig; es dauerte mehrere Wochen und erforderte die Hilfe meines Vaters. Deshalb war die Durchführung des Turniers im ursprünglich geplanten Umfang nicht möglich, da die Ferien schon begonnen hatten, als dieses Problem gelöst wurde. Jedoch wurde das Turnier mit meinen Geschwistern durchgeführt.

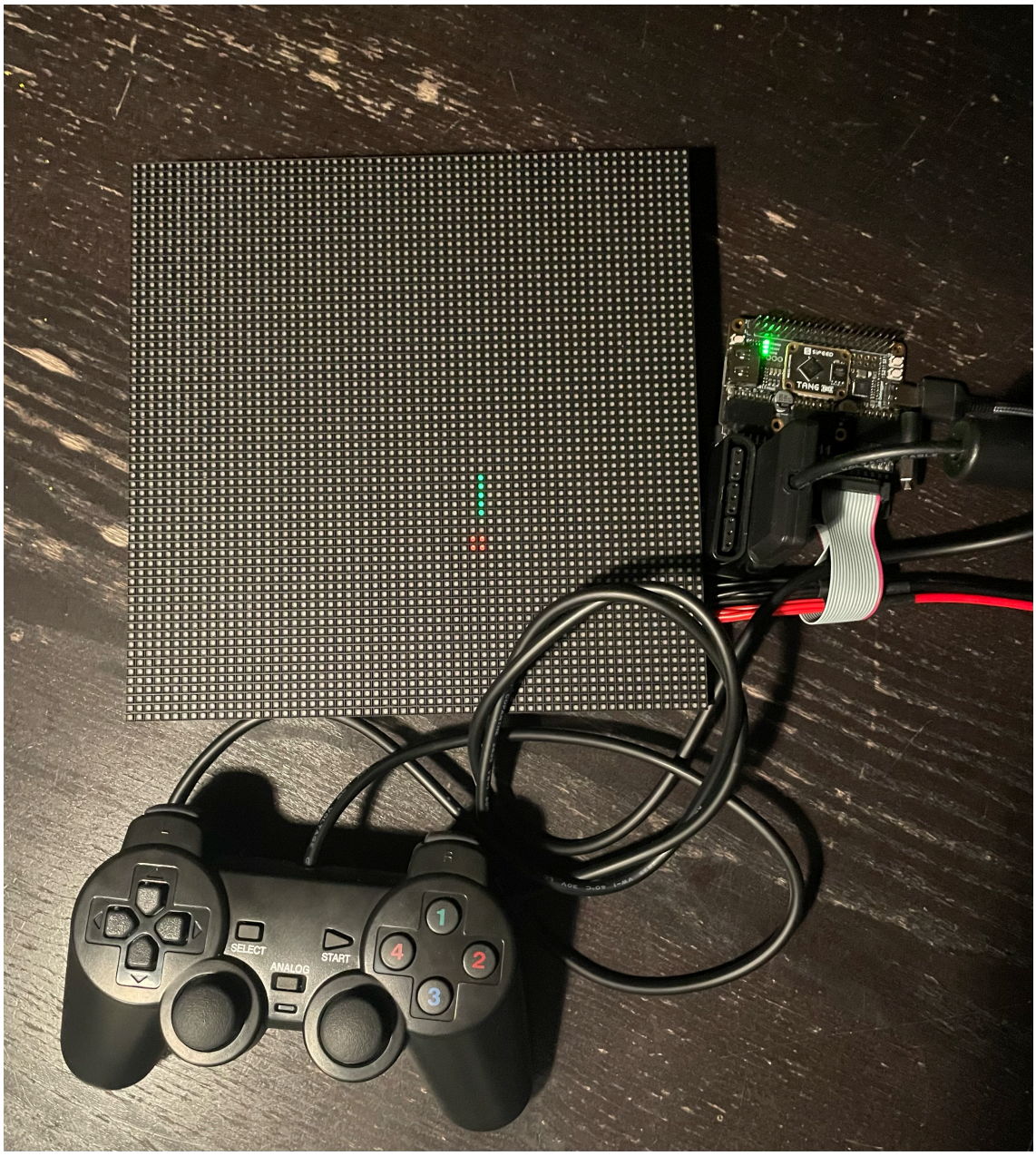

Abbildung 13: Das FPGA mit angeschlossenem Display und angeschlossenem Controller. Auf dem Display ist das Snake-Spiel zu sehen.

8. Fazit und Ausblick

In diesem Kapitel wird die Arbeit nochmals zusammengefasst und reflektiert. Anschließend folgen Ausblick und Erweiterungsmöglichkeiten. Abschließend folgt die Danksagung.

8.1. Zusammenfassung

Das Ziel der Arbeit war es, einen RISC-V-Prozessor mithilfe eines FPGAs zu entwerfen und zu implementieren, wobei anschließend geplant war, ein Snake-Spiel zu programmieren, um ein Turnier zu veranstalten. Zu Beginn wurde ein Emulator in C entwickelt, der das Verhalten des Prozessors simuliert und so ein grundlegendes Verständnis von der Prozessorarchitektur ermöglichte, ohne direkt mit FPGA-spezifischen Herausforderungen konfrontiert zu werden. Aufbauend auf den im Emulator gewonnenen Erkenntnissen folgten Entwurf und Implementierung des Prozessors als FSM in SystemVerilog. Im Anschluss an die erfolgreiche Umsetzung des Prozessors wurden Peripheriegeräte über Memory-Mapped I/O angebunden und mit in C programmierten Anwendungen integriert.

Um die Korrektheit und Funktionalität sowohl des Emulators als auch des Prozessors zu überprüfen, kamen verschiedene Test-Frameworks zum Einsatz. Die Validierung der Programme, des Emulators und des Prozessors erfolgte im Verlauf des Projekts. Nach Abschluss der Hardware- und Softwareintegration wurde das Snake-Spiel implementiert und das Turnier in eingeschränktem Umfang durchgeführt. Der Prozessor und der Emulator führen alle Instruktionen korrekt aus, zudem sind alle Peripheriegeräte funktionsfähig, und auch C-Programme werden nun zuverlässig ausgeführt.

8.2. Reflexion

Meine Motivation für dieses Projekt war die Frage, wie Computer auf elementarer Ebene funktionieren. Nach dem Abbruch der ursprünglichen Spielidee entschied ich mich daher bewusst für den Bau eines Prozessors, um diese Wissenslücke zu schließen. Jedoch unterschätzte ich zunächst die Komplexität des Themas.

Im Verlauf der Arbeit zeigte sich, dass mir Recherche, Dokumentation und konzentriertes Arbeiten nicht immer leichtfielen. Besonders die Implementierung und Fehlersuche in der Immediate-Dekodierungslogik waren eine große Herausforderung. Immer wieder traten Fehler auf, durch die Instruktionen nicht wie erwartet ausgeführt wurden. Die Ursachen lagen häufig in kleinen Details, deren Identifikation und Behebung viel Zeit und Geduld erforderten und teilweise externe Hilfe notwendig machten.

Zusätzlich verstärkte meine Tendenz zur Prokrastination den zeitlichen Druck, insbesondere gegen Ende der Abgabefrist. Aufgaben wurden teilweise zu spät angegangen, sodass ich manchmal meinen Schlaf opfern musste. Diese Arbeitsweise war nicht optimal, funktionierte aber trotzdem. Dies hat mir aber deutlich gezeigt, wie wichtig strukturiertes Vorgehen und Priorisierung sind.

Fachlich habe ich ein deutlich tieferes Verständnis dafür gewonnen, wie Instruktionen verarbeitet, Programme ausgeführt und im Speicher abgelegt werden und wie Software und Hardware auf elementarer Ebene zusammenwirken. Besonders bedeutsam ist für mich, einen funktionsfähigen Prozessor von Grund auf entworfen und umgesetzt zu haben. Trotz der Schwierigkeiten war das Projekt insgesamt sehr lehrreich. Ich konnte Wissenslücken schließen und mein Interesse an Rechnerarchitektur weiter vertiefen. Zudem habe ich gelernt, dass ich um Hilfe bitten kann, und sie auch erhalte.

8.3. Ausblick und Erweiterungsmöglichkeiten

Der im Rahmen dieser Arbeit entworfene und implementierte Prozessor hat noch einige Verbesserungsmöglichkeiten. Es können noch weitere Erweiterungen implementiert werden, zum Beispiel die M-Erweiterung, welche Multiplikations- und Divisionsbefehle hinzufügt. Außerdem könnte noch ein Betriebssystem implementiert werden. Darüber hinaus könnten noch weitere Peripheriegeräte angebunden werden, wie zum Beispiel eine Tastatur oder eine Maus. Die Modularität von RISC-V lässt es hier zu, dass viele dieser Änderungen ohne signifikante Codeänderungen vorgenommen werden können. Ich werde dieses Projekt auch in meiner Freizeit noch weiterführen und verschiedene Erweiterungen umsetzen.

8.4. Danksagung

Ich danke meiner Mentorin Frau Michaelis für die hilfreichen Gespräche und die Abgabetermine. Ich danke meinem Vater, ohne den ich nicht auf die Idee für das Projekt gekommen wäre und ohne dessen Hilfe diese Arbeit nur halb so umfangreich wäre. Ich danke meiner Mutter für die regelmäßigen Produktivitätskontrollen, ohne die die Arbeit deutlich langsamer vorangeschritten wäre. Ich danke meinen Geschwistern Erik und Gian für die Teilnahme am „Snake-Turnier“.

9. Glossar

Array: Ein Array ist eine sequenzielle Datenstruktur, die eine feste Anzahl von Elementen eines bestimmten Datentyps enthält und über einen Index zugänglich ist. 10

Bug: Bug ist eine Bezeichnung für Fehler im Programmcode, sie stammt aus der Zeit, in der Computer noch Räume ausfüllten und Insekten im Computer Probleme verursachten und wurde später als allgemeiner Begriff für Fehler in Software oder Hardware übernommen. 11

***CPU* – Central Processing Unit**: Die zentrale Recheneinheit eines Computers, die Befehle ausführt und Daten verarbeitet. 3

***Creative Commons* – Creative-Commons-Lizenzierung**: Eine Lizenzierungsoption, die es Urhebern ermöglicht, ihre Werke unter bestimmten Bedingungen freizugeben und anderen die Nutzung zu erlauben. 4

Kontrollflussinstruktion: Kontrollflussinstruktionen beeinflussen, in welcher Reihenfolge die Instruktionen ausgeführt werden. Sprünge und Bedingungen sind solche Instruktionen. 9

Emulator: Software, die das Verhalten von Hardware nachahmt. 5

Endianness: Endianness beschreibt die Anordnung von Bytes in einem mehrbyteigen Datentyp, insbesondere wie die Bytes in Bezug auf ihre Bedeutung angeordnet sind. 12

Execution Tracing: Execution Tracing beschreibt einen Prozess, bei dem während der Programmausführung Informationen über dessen internen Zustand gesammelt werden, um Fehler zu finden. 11, 11, 11

***FPGA* – Field Programmable Gate Array**: Ein rekonfigurierbarer integrierter Schaltkreis, der aus programmierbaren Logikeinheiten besteht und zur Implementierung digitaler Schaltungen verwendet werden kann. 15

***FSM* – Finite State Machine** 12, 12

Host: Im Kontext des Emulators ist der Host das physische System, auf dem der Emulator ausgeführt wird. 9

***ISA* – Instruction Set Architecture**: Die ISA definiert, welche Befehle einem Prozessor zur Verfügung stehen und welche Funktionen diese ausführen. 4, 6

Little Endian: Ein Endianness-Format, bei dem die niederwertigsten Bytes eines mehrbyteigen Datentyps an der niedrigsten Adresse gespeichert werden. 12

***LSB* – Least Significant Byte**: Das Least Significant Byte (LSB) ist das Byte mit der geringsten Wertigkeit in einem mehrbyteigen Datentyp. 12

***MMIO* – Memory Mapped I/O**: Ein Konzept, bei dem Peripheriegeräte über den Adressraum des Hauptspeichers angesprochen werden, anstatt über spezielle Ein- und Ausgabebefehle. 5

***MSB* – Most Significant Byte**: Das Most Significant Byte (MSB) ist das Byte mit der höchsten Wertigkeit in einem mehrbyteigen Datentyp. 12

***Open Source* – Open Source Software**: Software, deren Quellcode öffentlich zugänglich ist und von jedem eingesehen, verändert und weiterverbreitet werden kann. 4

***PC* – Program Counter**: Der Program Counter ist die Adresse der nächsten Instruktion, die ausgeführt wird. 9, 10

Peripheriegerät: Externe Geräte, die an einen Computer angeschlossen werden, um zusätzliche Funktionen bereitzustellen, wie z. B. Displays, Joysticks oder Tastaturen. 5

Pipelining: Pipelining ist eine Technik, bei der mehrere Instruktionen gleichzeitig in verschiedenen Stufen verarbeitet werden, was die Effizienz des Prozessors erhöht. 9

Rechnerarchitektur: Ein Teilgebiet der Informatik, das sich mit dem Aufbau und der Funktionsweise von Computersystemen befasst. 4

Snake: Ein einfaches Videospiel, bei dem der Spieler eine Schlange steuert, die sich auf einem Spielfeld bewegt und dabei Punkte sammelt, indem sie Nahrung frisst. 3

Taktzyklen: Die Anzahl der Taktzyklen, die eine Instruktion benötigt, um vollständig ausgeführt zu werden. 9

Unit-Test: Ein Unit-Test ist eine Methode der Softwareentwicklung, bei der einzelne Komponenten oder Funktionen eines Programms isoliert getestet werden, um sicherzustellen, dass sie korrekt funktionieren. 11, 11

Wort: Ein Wort ist eine Datenmenge, die von einem Prozessor in einem einzigen Schritt verarbeitet werden kann. In diesem Projekt ist ein Wort 32 Bit breit. 10

Quellen

[1] J. L. Hennessy und D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th Edition. San Francisco, CA, USA: Morgan Kaufmann, 2019.

[2] D. A. Patterson und J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, Second. Morgan Kaufmann/Elsevier, 2020.

[3] E. Cui, T. Li, und Q. Wei, „RISC-V Instruction Set Architecture Extensions: A Survey“, *IEEE Access*, Bd. 11, S. 24696–24711, 2023, doi: 10.1109/ACCESS.2023.3246491.

[4] R. Johnson, *RISC-V Architecture and Implementation Guide: Definitive Reference for Developers and Engineers*. Hitex Press, 2025.

[5] D. Patterson und A. Waterman, *The RISC-V Reader: An Open Architecture Atlas*. Strawberry Canyon, 2017.

[6] W. Gay, *RISC-V Assembly Language Programming: Using ESP32-C3 and QEMU*. Elektor, 2022.

[7] S. Smith, *RISC-V Assembly Language Programming*. Apress, 2024. doi: 10.1007/979-8-8688-0137-2.

[8] A. Shriram, „RISC-V Reference“. [Online]. Verfügbar unter: https://www.cs.sfu.ca/~ashriram/Courses/CS295/assets/notebooks/RISCV/RISCV_CARD.pdf

[9] A. Waterman und K. Asanović, „The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 20191214-draft“. [Online]. Verfügbar unter: https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf